

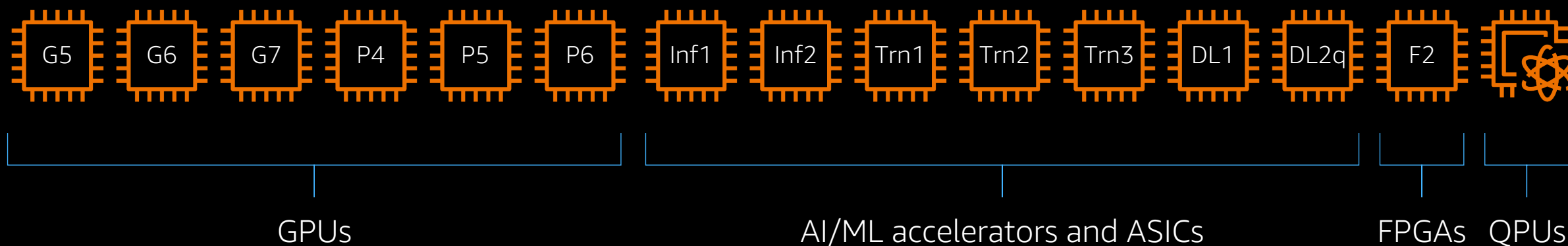
Amazon Braket

Quantum computing on AWS



Vision: Quantum as foundational compute technology

Part of our accelerated computing portfolio



Trainium accelerator
Inferentia accelerator



GB200, B200, H200, H100,
A100, V100,
A10G, T4



Gaudi accelerator

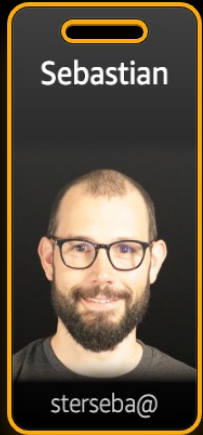


Radeon GPU
Xilinx accelerator
Xilinx FPGA



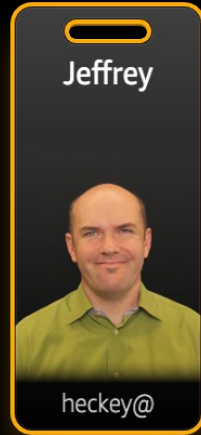
Qualcomm Cloud
AI 100





Sebastian Stern

Sr. WW Specialist SA Quantum
WWSO Advanced Compute
AWS



Jeff Heckey

Sr. SDE
Amazon Braket
AWS

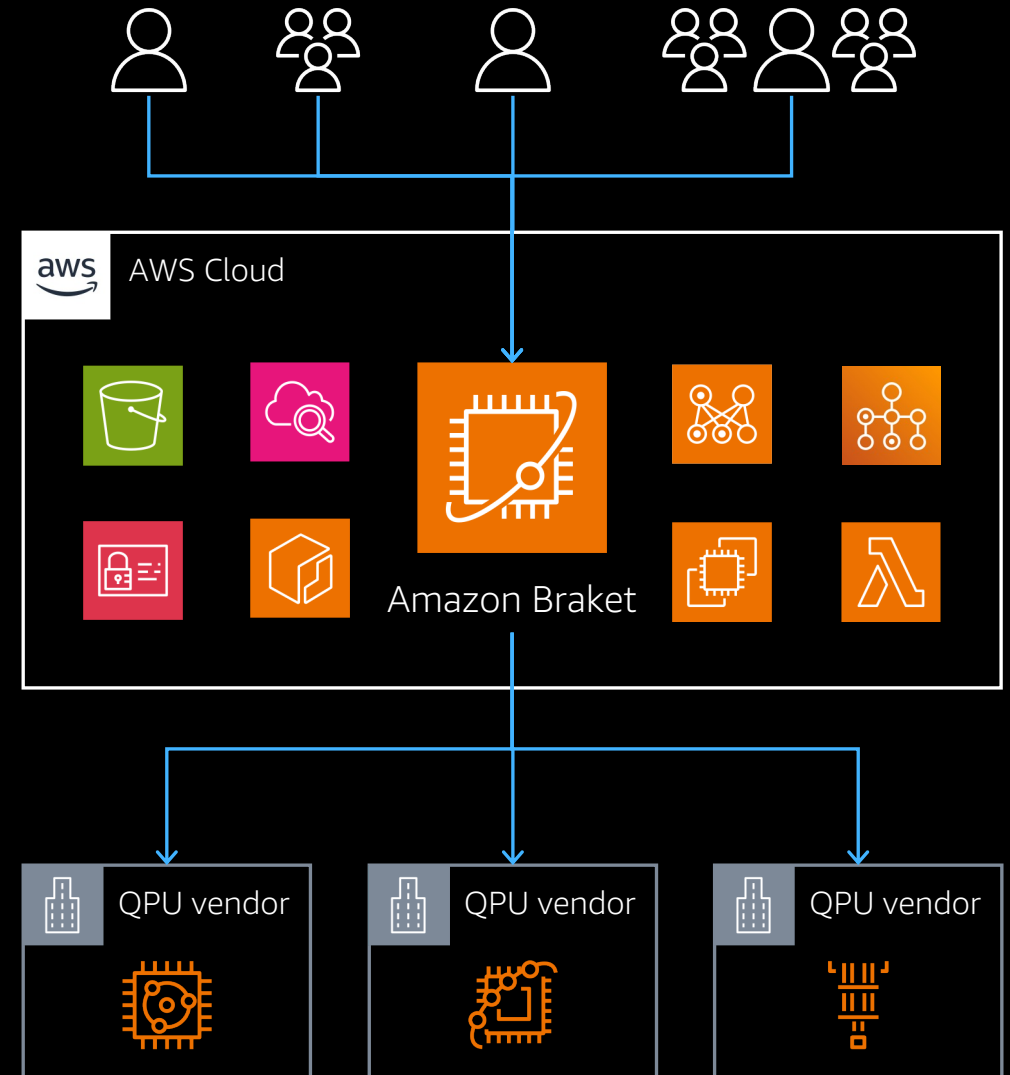
Agenda

1. Introduction to Amazon Braket
2. Amazon Braket internals
3. Integration of quantum and classical computing resources

Amazon Braket

AWS quantum computing cloud service

- On-demand & dedicated access to quantum hardware from various providers
- Unified user experience across device modalities
- Low-level access and device-native programming
- Fully-managed execution environment for quantum-classical workloads
- AWS cloud integration



Quantum hardware integrated over time



Device Integration - HAPI

Hardware-provider API

- Wrapper interface to Braket's execution workflow
- Device agnostic
- Directly compatible with
 - REST APIs
 - gRPC
 - Python libraries
- ABIs can be attached with pybind
- Natively integrates into AWS Cloud Services

```
# Security
def use_credential(...):
    """ Uses current credential """
def new_api_credential(...) -> str:
    """ Create a new credential for api """
def new_web_credential(...) -> Optional[str]:
    """ Create a new web credential """

# Device management
def get_device_status(...) -> DeviceStatus:
    """ Get the status of a device """
def get_device_calibration(...) -> DeviceCalibration:
    """ Get the calibration of a device """
def get_device_availability_calendar(...) -> Optional[str]:
    """ Get device availability """

# Task execution
def create_task(...) -> Task:
    """ Create a task """
def get_task(...) -> Task:
    """ Get a task """
def cancel_task(...) -> CancelTaskStatus:
    """ Cancel a task """
def create_task_parameters(...) -> Optional[TaskParameters]:
    """ Create task parameters for a device """
def transform_to_runnable(...) -> Optional[Runnable]:
    """ Transform the program to a runnable format """
```

How users interact with Braket

Service APIs



Quantum Tasks

Run quantum programs on on-demand simulators or QPUs



Braket Hybrid Jobs

Run hybrid programs combining quantum and classical computing resources

AWS SDKs

Call Amazon Braket from within your applications



Python



C++



Rust

...and more programming languages

Programming Frameworks



Amazon Braket Python SDK



PennyLane



Qiskit

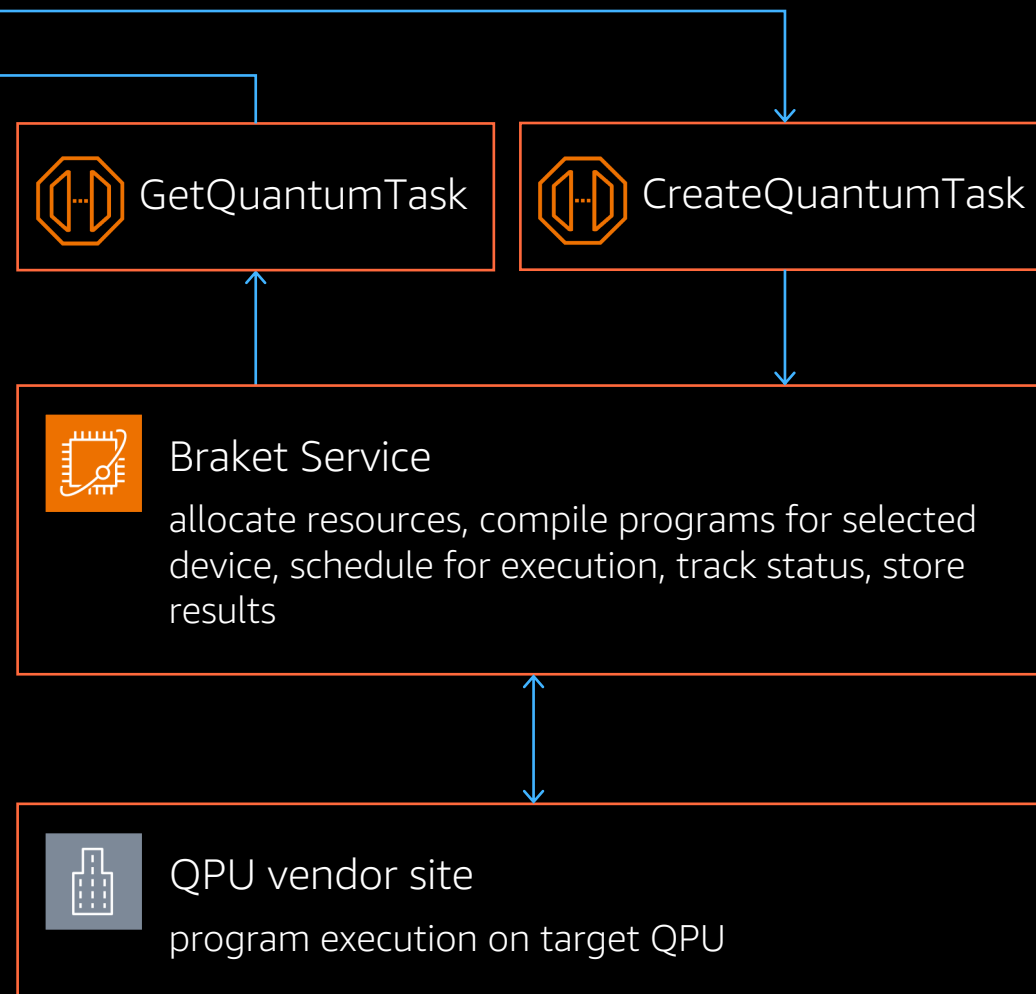


CUDA-Q

...and more 3rd-party tools & programming frameworks supported

Run logical quantum programs with Braket

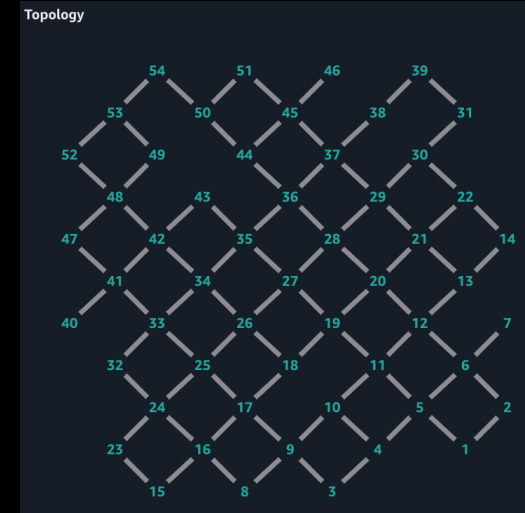
```
~ pip install amazon-braket-sdk
~
~ python
>>> from braket.aws import AwsDevice
>>> from braket.devices import Devices
>>> from braket.circuits import Circuit
>>>
>>> circuit = Circuit().h(0).cnot(0,1)
>>>
>>> qpu = AwsDevice(Devices.IQM.Garnet)
>>>
>>> task = qpu.run(circuit, shots=100)
>>> task.state()
'QUEUED'
>>> task.queue_position()
QuantumTaskQueueInfo(queue_type='Normal', queue_position='1')
>>> task.state()
'COMPLETED'
>>>
>>> result = task.result()
>>> result.additional_metadata.iqmMetadata.compiledProgram
'OPENQASM 3.0;\nbit[2] c;\n#pragma braket verbatim\nbox
{\nprx(0.5*pi,1.5*pi) $49;\nprx(0.5*pi,1.5*pi) $53;\nncz
$49,$53;\nprx(0.5*pi,0.5*pi) $53;\n}\nnc[0] = measure
$49;\nnc[1] = measure $53;'
```



Low-level access and device-native programming

```
>>> qpu = AwsDevice(Devices.IQM.Garnet)
>>>
>>> qpu.properties.paradigm.nativeGateSet
['cz', 'prx', 'cc_prx', 'measure_ff']
>>>
>>> circuit = Circuit().prx(1,0.5*pi,1.5*pi)
>>> circuit.prx(2,0.5*pi,1.5*pi).cz(1,2).prx(2,0.5*pi,0.5*pi)
>>> verbatim_circuit = Circuit().add_verbatim_box(circuit)
>>> task = qpu.run(verbatim_circuit, shots=100,
                  disable_qubit_rewiring=True)
```

```
>>> from braket.pulse import PulseSequence, GaussianWaveform
>>>
>>> qpu = AwsDevice(Devices.Rigetti.Ankaa3)
>>>
>>> drive_frame = device.frames["Transmon_25_charge_tx"]
>>> readout_frame = device.frames["Transmon_25_readout_rx"]
>>> waveform = GaussianWaveform(FreeParameter("length"),
                                FreeParameter("length")*0.25, 0.2, False)
>>> pulse_sequence = PulseSequence().play(drive_frame,
                                           waveform).capture_v0(readout_frame)
>>> task = qpu.run(pulse_sequence(length=12e-9), shots=100)
```



Calibration

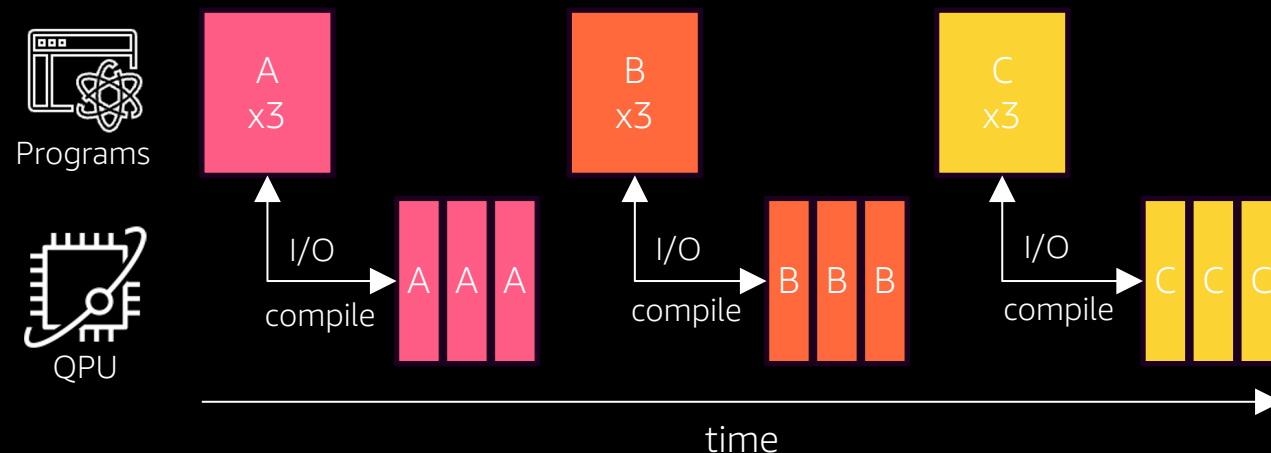
Last updated: Feb 11, 2026 07:13 (UTC) Info

Qubit

Qubit	T1 (μs) Info	T2 (μs) Info	Fidelity (simultaneous RB) (%) Info	Readout fidelity (%) Info
1	42.757 ± 2.065	14.121 ± 0.750	99.917 ± 0.004	98.150 ± 0.215
2	48.633 ± 3.370	25.881 ± 1.884	99.931 ± 0.002	98.400 ± 0.200
3	44.152 ± 1.676	34.356 ± 2.157	99.946 ± 0.001	98.925 ± 0.161
4	33.283 ± 2.528	10.366 ± 0.681	99.928 ± 0.002	98.075 ± 0.217
5	51.002 ± 2.189	22.013 ± 1.415	99.956 ± 0.002	95.575 ± 0.325
6	37.210 ± 1.901	12.036 ± 0.699	99.937 ± 0.002	98.475 ± 0.193
7	43.815 ± 4.111	18.113 ± 0.953	99.923 ± 0.004	96.250 ± 0.298
8	44.356 ± 3.592	32.902 ± 1.724	99.849 ± 0.006	98.475 ± 0.191
9	28.096 ± 4.511	8.408 ± 1.052	99.762 ± 0.006	98.275 ± 0.204
10	49.751 ± 3.919	20.765 ± 1.586	99.962 ± 0.002	97.800 ± 0.231
11	39.227 ± 5.261	8.816 ± 0.581	99.944 ± 0.002	97.875 ± 0.233
12	48.535 ± 4.750	16.172 ± 0.894	99.969 ± 0.001	98.525 ± 0.190
13	38.915 ± 1.875	10.451 ± 0.680	99.933 ± 0.002	97.575 ± 0.247
14	60.262 ± 3.056	19.786 ± 0.944	99.962 ± 0.001	98.350 ± 0.205

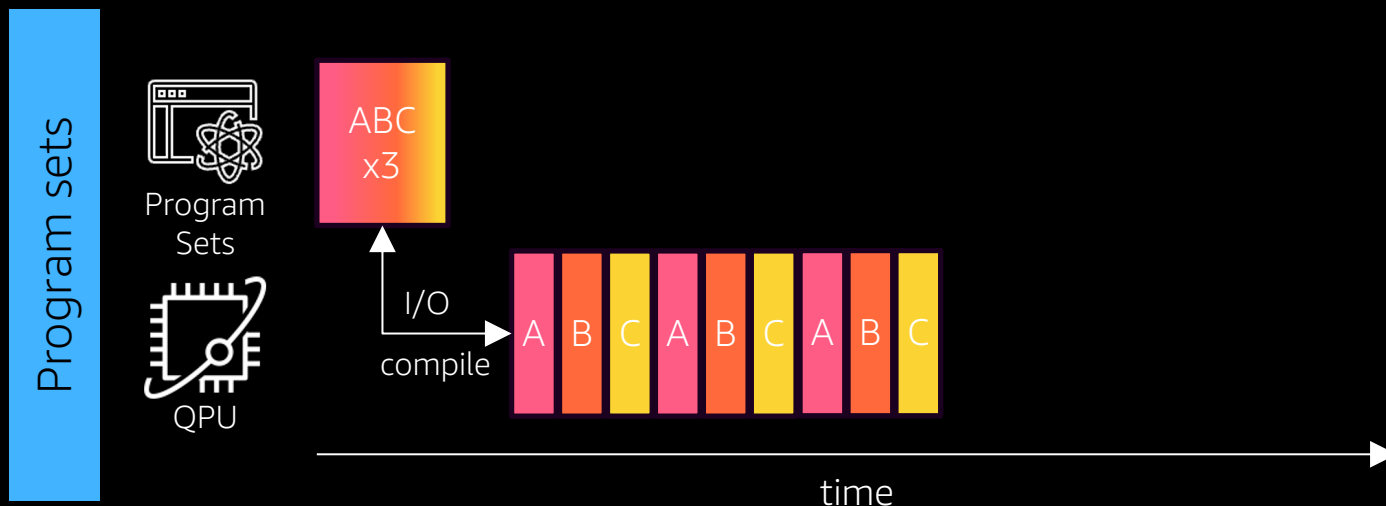
- Access to device properties: qubit count, native gates, connectivity, calibration data...
- Program verbatim circuits executed as defined, without modifications
- Access pulse control on certain devices and define your programs analog pulse sequences

Bracket program sets – efficient batch execution



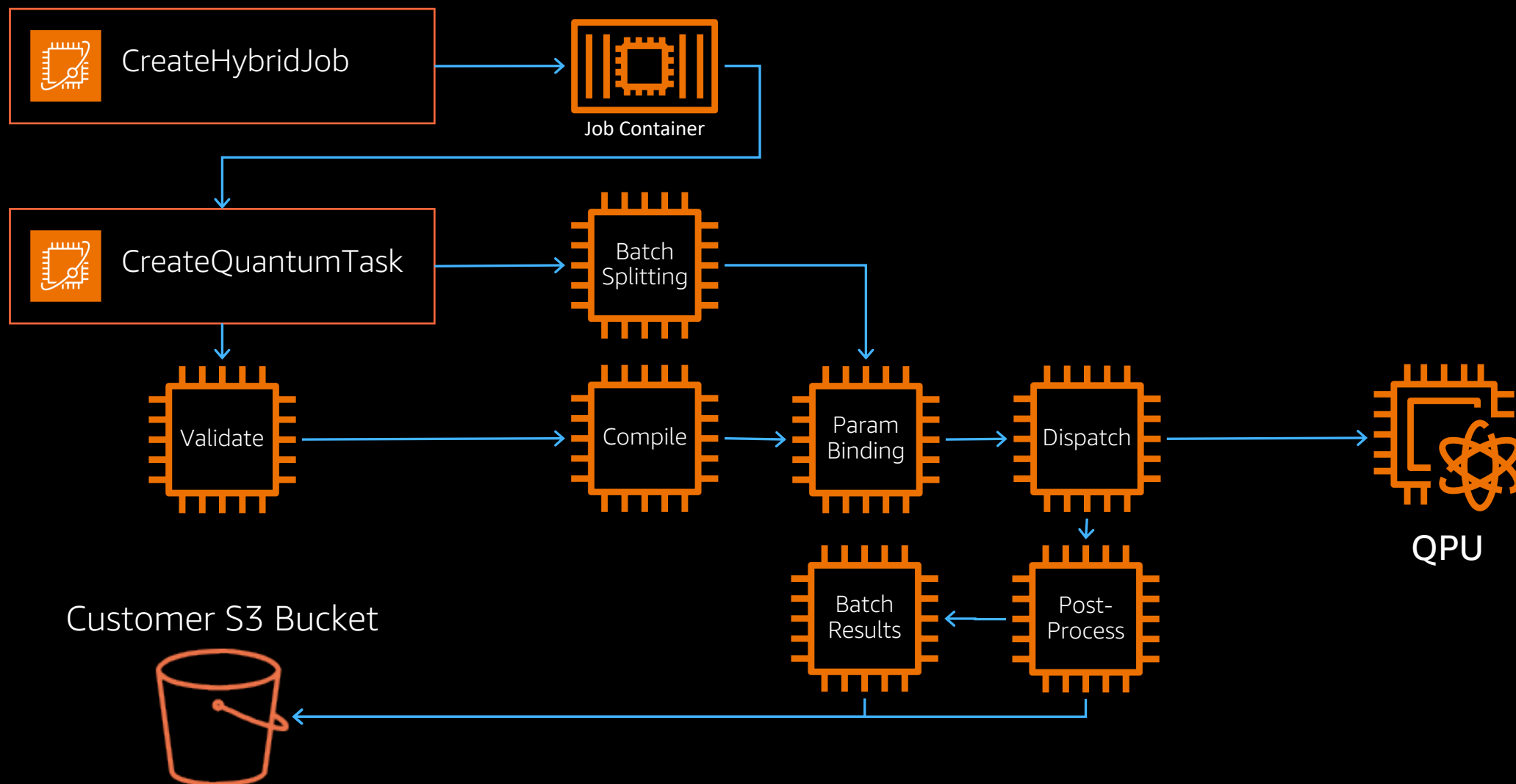
Common workload pattern for

- Hamiltonian estimation
- parameter sweeps
- gradient computations
- ...

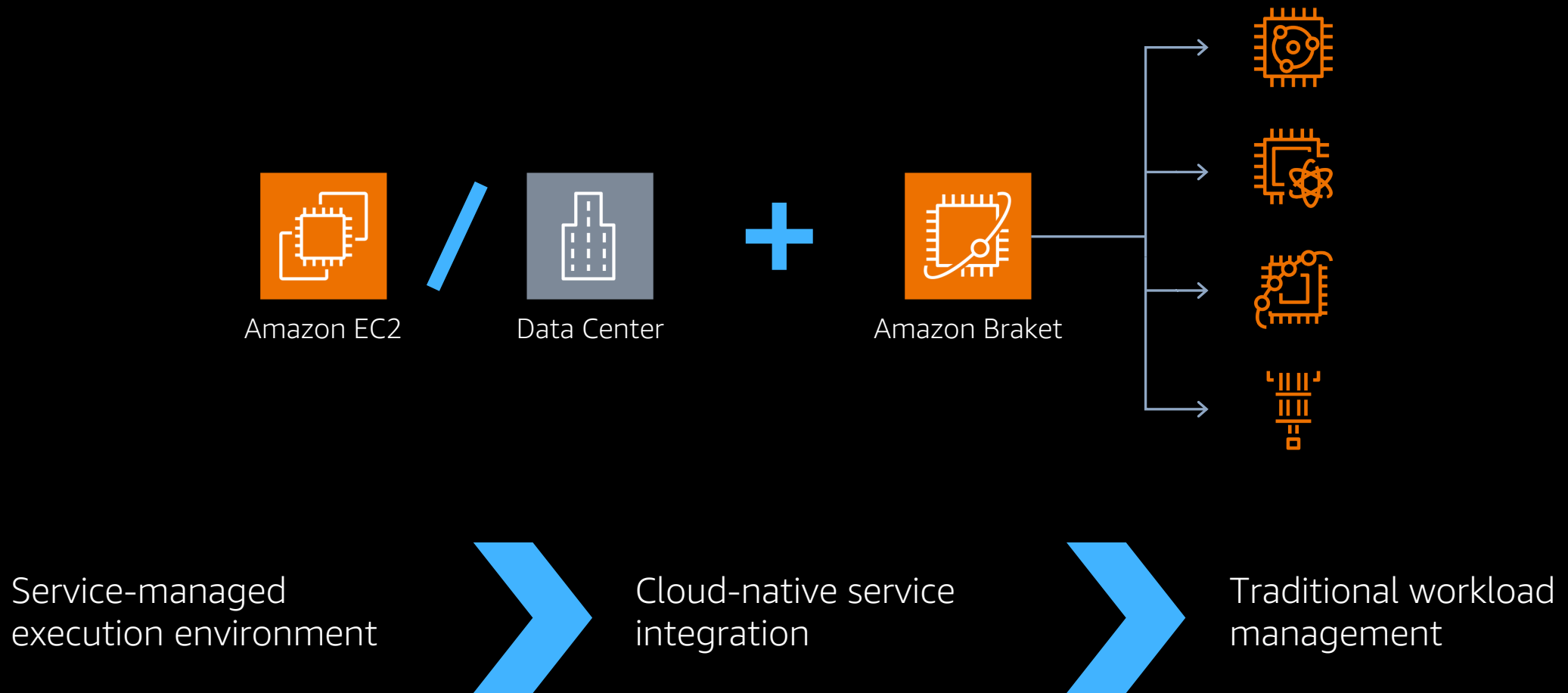


- ✓ Submit multiple quantum programs in a single payload
- ✓ Compiled to a hardware efficient representation
- ✓ End-to-end execution with up to 24x speedup

Braket Execution Flow



Integration of quantum and classical resources



Amazon Braket Hybrid Jobs

Fully-managed execution environment that makes it [easy](#) to run quantum-classical workloads on Braket

```
>>> import pennylane as qml
>>> from braket.jobs import hybrid_job
>>>
>>> @hybrid_job(device=Devices.IonQ.Arial, local=False)
>>> def qubit_rotation(steps=20, step_size=0.5, shots=1000):
...     device = qml.device("braket.aws.qubit", device_arn=get_job_device_arn(), wires=1, shots=shots)
...
...     @qml.qnode(device)
...     def circuit(params):
...         qml.RX(params[0], wires=0)
...         return qml.expval(qml.PauliZ(0))
...
...     opt = qml.GradientDescentOptimizer(step_size)
...     for i in range(steps):
...         params, expval = opt.step_and_cost(circuit, params)
...
...     return {"theta": params.tolist()[0]}
>>>
>>> job = qubit_rotation()
```

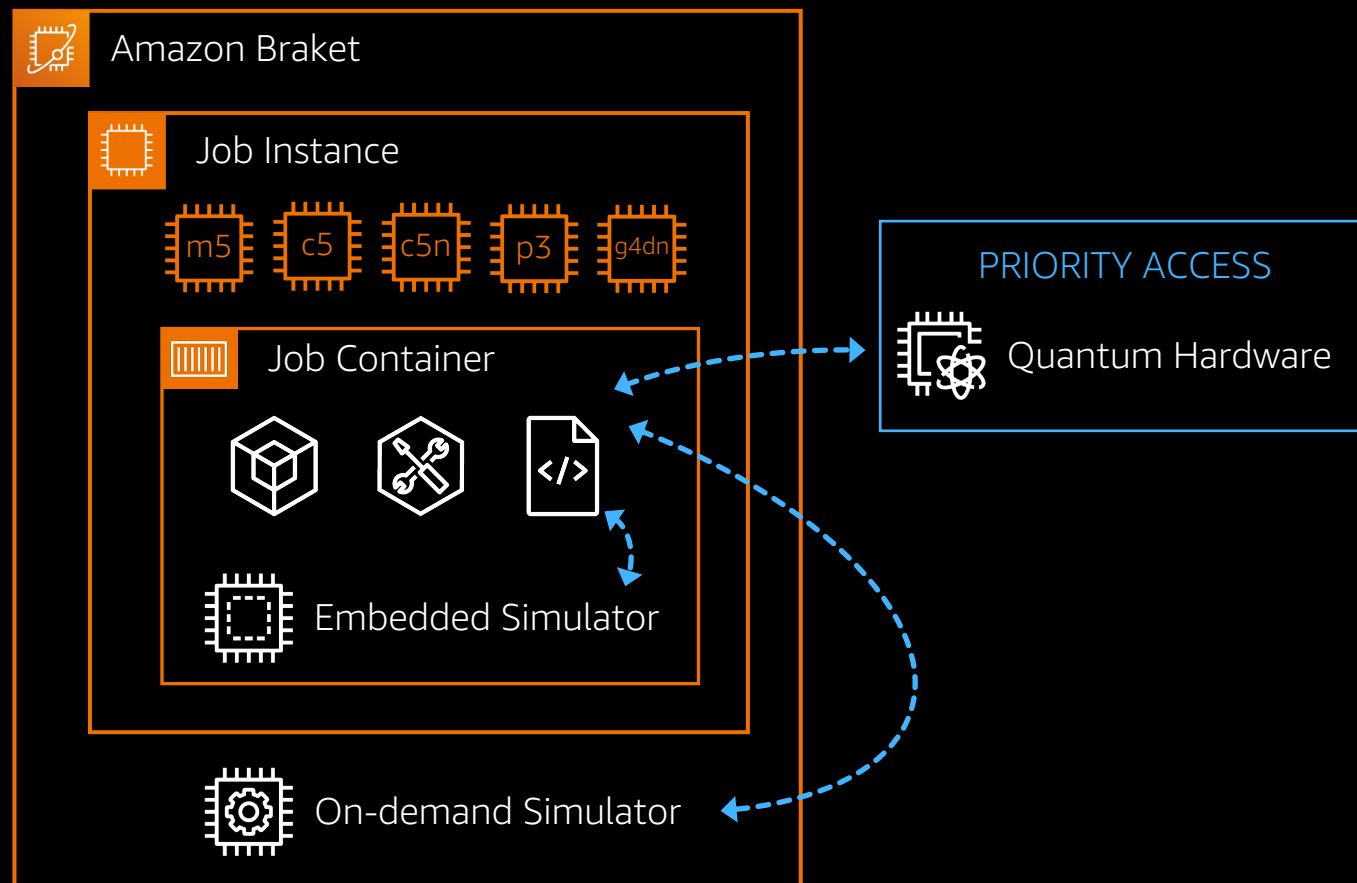
Amazon Braket Hybrid Jobs nuts and bolts

```
from braket.aws import AwsQuantumJob  
job = AwsQuantumJob.create(...)
```

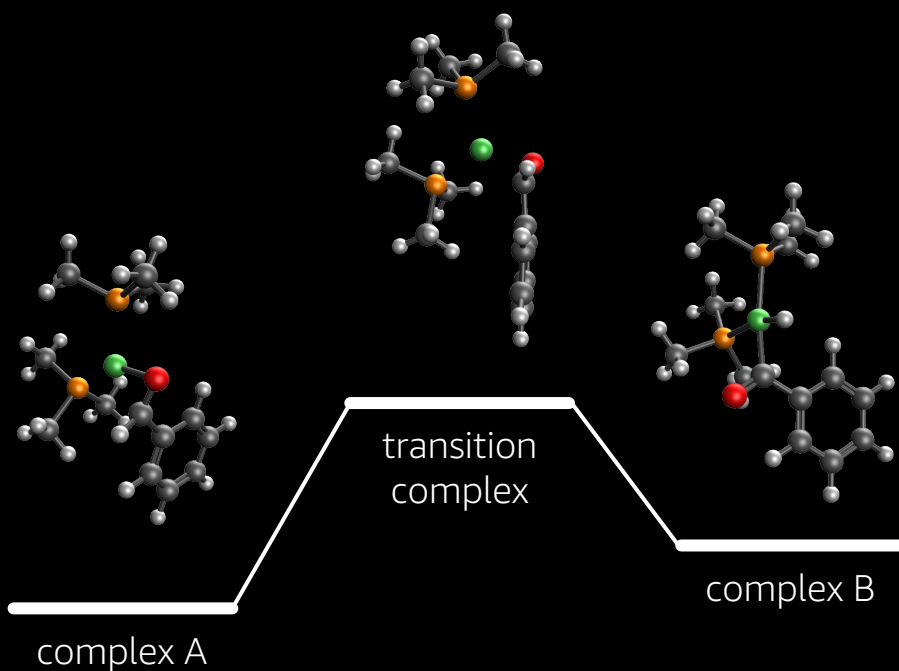


CreateJob API

- Start and provision **Job Instance**: user-selected classical compute resources
- Run **Job Container**: based on pre-built or user-provided Docker image
- Execute algorithm code
- Run tasks on QPUs, on-demand simulators, or simulators embedded into job container
- Release compute resources at job completion



Quantum-HPC Monte Carlo Simulations



- Modeling of the Suzuki-Miyaura reaction
- Quantum-classical quantum Monte Carlo algorithm
- Executed on IonQ's Forte quantum computer via Amazon Braket
 - Over 60k quantum circuits with 24 qubits
- Post-processing with AWS ParallelCluster
 - 320 NVIDIA H200 GPUs (EC2 P5en instances)

Hybrid Auxiliary Field Quantum Monte Carlo

Trial wavefunction

Quantum computation

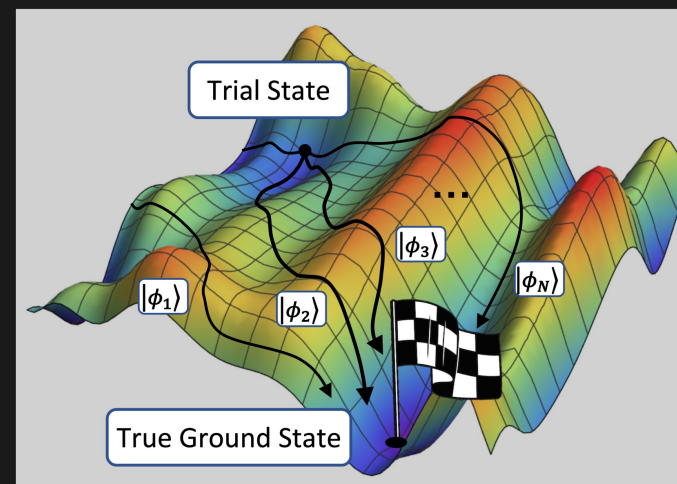
$$\begin{bmatrix} 1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 1 \end{bmatrix}$$



Amazon Braket

Evolve walkers, evaluate energy

Classical post-processing



B. Huang. Phys. Rev. Research 6, (2024) 043063



AWS ParallelCluster



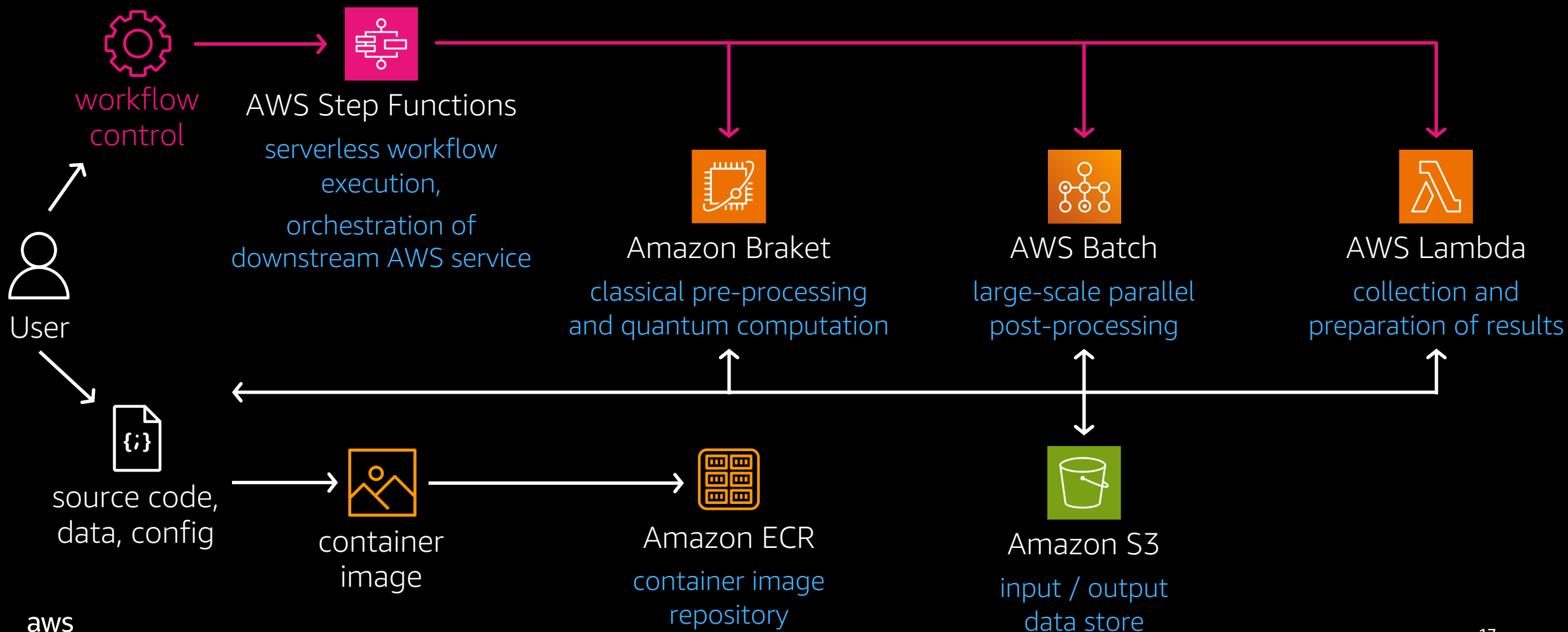
AWS PCS



AWS Batch

Cloud-native compute pipeline

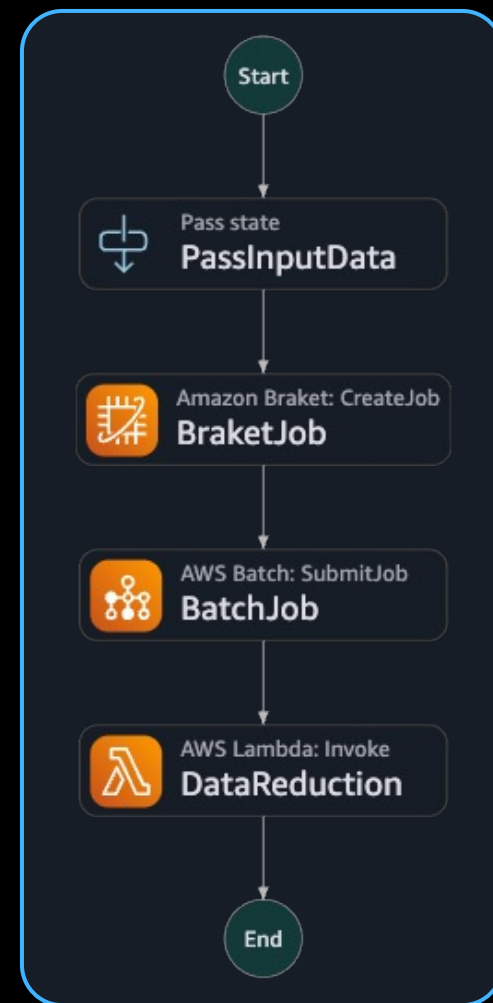
For workflows with classical pre- / post-processing at HPC scale



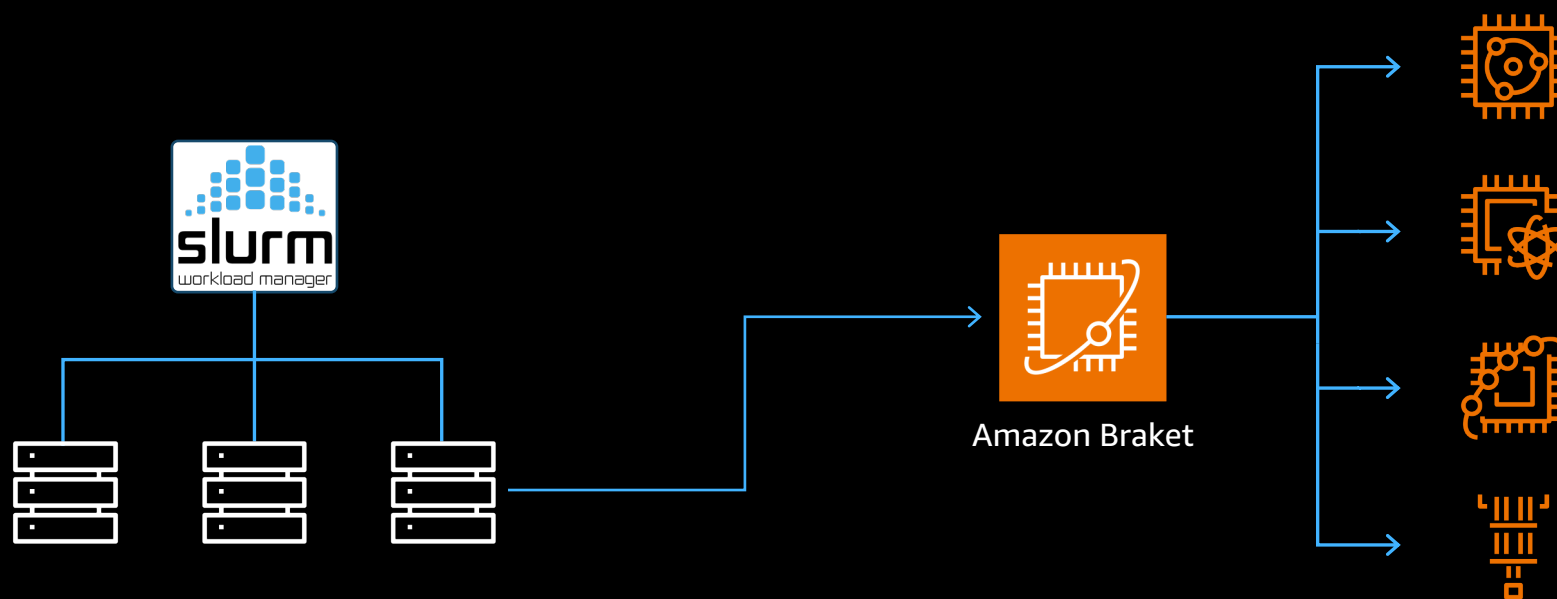
Deploy and run the hybrid workflow

```
AWSTemplateFormatVersion: 2010-09-09
Resources:
  HybridWorkflowOrchestration:
    Type: AWS::StepFunctions::StateMachine
    Properties:
      RoleArn: !ImportValue BraketBatchWorkflowRoleArn
      Definition:
        QueryLanguage: JSONata
        StartAt: PassInputData
        States:
          BraketJob:
            Type: Task
            Resource: arn:aws:states:::aws-
sdk:braket:createJob.waitForTaskToken
            Arguments:
              AlgorithmSpecification:
                ScriptModeConfig:
                  EntryPoint: afqmc.collect_shadow:main
                  S3Uri: !Sub s3://${DataBucket}/afqmc.tar.gz
                  CompressionType: GZIP
              DeviceConfig:
                Device: local:pennylane/lightning.qubit
              HyperParameters:
                TaskToken: "{% $states.context.Task.Token %}"
                ShadowSize: "{% $ShadowSize %}"
                Shots: "{% $Shots %}"
              InstanceConfig:
```

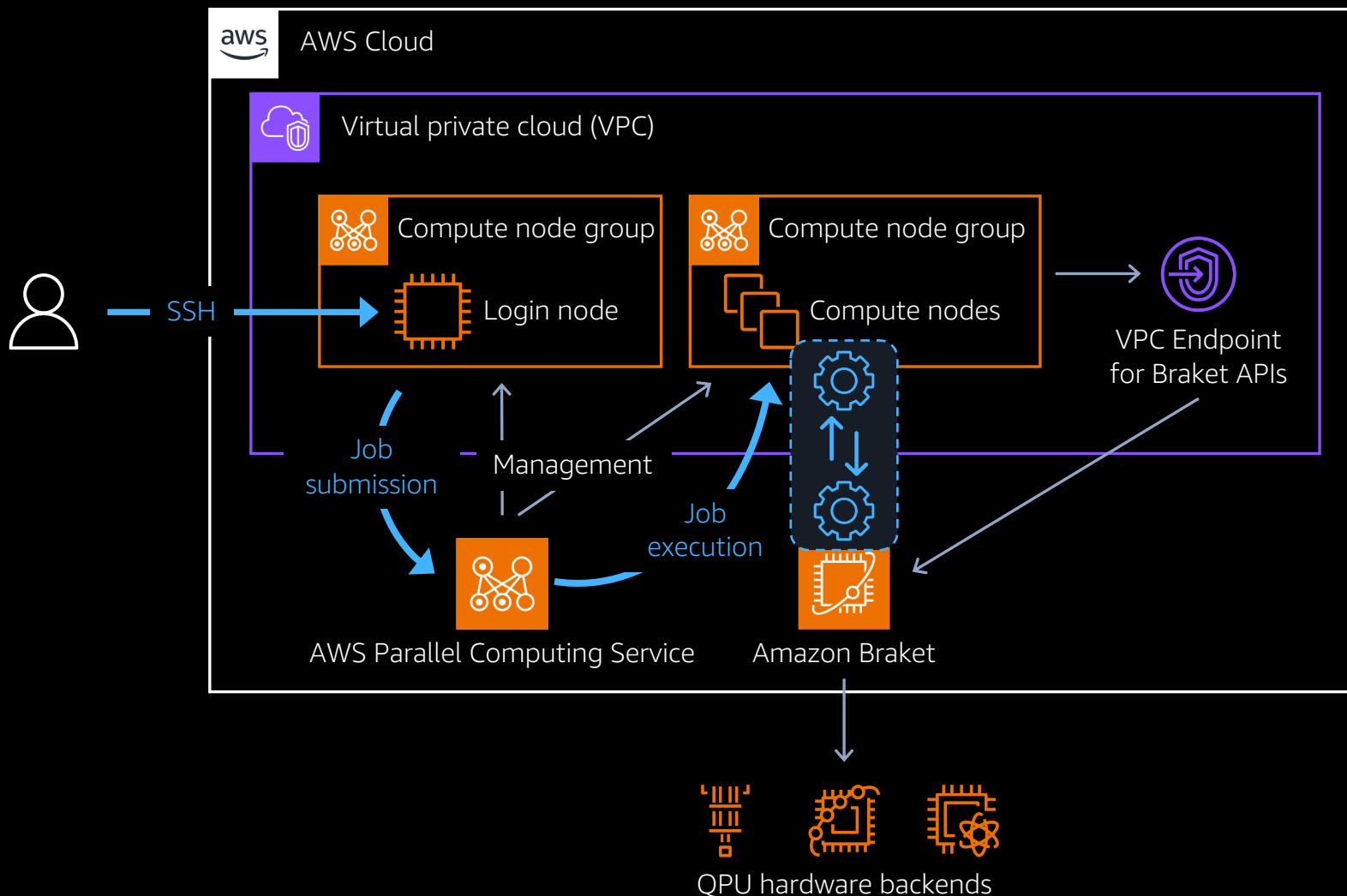
- Describe workflow with IaC
- Deploy with an API call
`CloudFormation::CreateStack`
- Run full simulation with an API call
`StepFunctions::StartExecution`
- Download results from Amazon S3
- Develop, maintain, and share experiment including infrastructure setup and application code in [one repository](#)



Access to Braket from traditional HPC environments



Quantum-accelerated Slurm partitions with Braket



Thoughts on resource scheduling

Option 1: Dedicated QPU access

- Device reservations give exclusive access to a QPU with exact start and end time
- Slurm resource reservations to reserve classical compute capacity in same timeframe

Option 2: On-demand QPU access

- Braket GetDevice API contains availability and queue depth information
- Spank plugin to configure job execution depending on QPU availability and utilization

Thank you!

